

Hotel Booking Management System

1. Title Page:

INSTITUTE OF ENGINEERING AND MANAGEMENT

COURSE NAME: ADVANCED PROGRAMMING (OOP)

PROJECT TITLE: HOTEL BOOKING MANAGEMENT SYSTEM

STUDENT NAME: SOUMI CHAKRABORTY

ROLL NUMBER: 56

ENROLLMENT NUMBER: 12024052017058

**INSTRUCTOR NAME: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip
Biswas**

SEMESTER: SPRING 2026

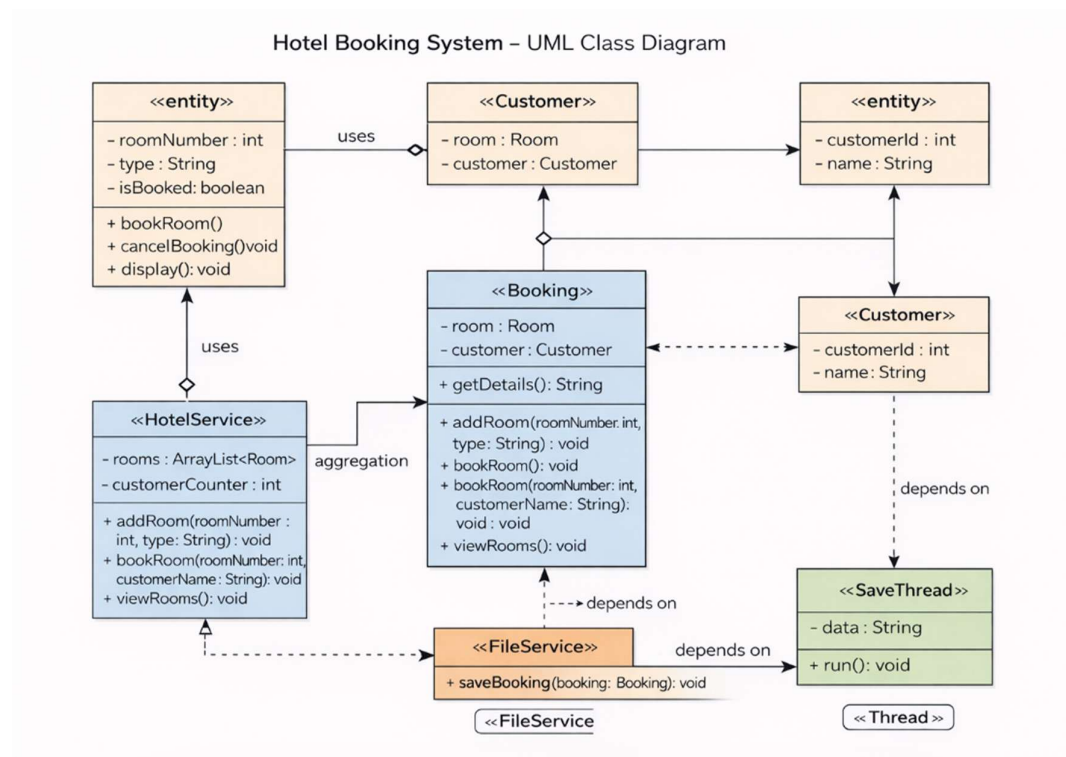
2. Problem Statement:

Managing hotel bookings manually is inefficient and error-prone, especially when handling multiple rooms and customers. Issues such as double bookings, incorrect availability tracking, and poor customer data management frequently occur. As hotel operations grow, maintaining accurate and persistent records becomes difficult.

The proposed Hotel Booking System provides an automated solution using Java. It allows users to add rooms, check availability, book and cancel reservations, and manage customer details efficiently. The system includes data persistence using file handling to retain booking records and uses multithreading to perform background operations such as saving data without interrupting the user interface. By applying Object-Oriented Programming principles and modular design, the system improves reliability, scalability, and usability for hotel management.

3. System Design:

3.1 UML Class Diagram:



3.2 Description of Major Classes:

- **Room:** Represents hotel rooms with attributes like room number, type, and booking status. Includes methods to book and cancel rooms.
- **Customer:** Stores customer details such as ID and name.
- **Booking:** Connects a Room with a Customer and stores booking details.

- **HotelService:** Acts as the core controller managing rooms, bookings, and customer interactions.
- **FileService:** Handles file operations to save booking data for persistence.
- **SaveThread:** Extends Thread to perform background saving operations.
- **HotelGUI:** Provides a graphical interface for user interaction using Java Swing.

3.2 Package Structure:

The project follows a modular package structure to organize code efficiently.

Main Package (Main)

- Contains the entry point of the application (Main.java).
- Responsible for starting the program and launching the GUI or console interface.

Model Package (Model)

- Contains core data classes such as Room, Customer, and Booking.
- Represents the structure of data used in the system.
- Encapsulates attributes and basic methods related to entities.

Service Package (Service)

- Contains business logic classes like HotelService, FileService, and SaveThread.
- Handles operations such as adding rooms, booking, cancellation, and data persistence.
- Acts as a bridge between the Model and View layers.

View Package (view)

- Contains GUI-related classes such as HotelGUI.
- Responsible for user interaction using Java Swing components.
- Sends user actions to the Service layer and displays results.

3.3 Interaction Flow:

- User interacts with GUI
- GUI sends request to HotelService
- Service processes logic using Model classes
- Booking is created and stored
- SaveThread saves data in background
- Output is displayed to the user

4. OOP Concepts Demonstrated:

4.1 Encapsulation:

Where: Room, Customer classes

Why: Protects internal data

Snippet:

```
        private int roomNumber;

    public int getRoomNumber() {
        return roomNumber;
    }
```

Explanation: Data is kept private and accessed through methods, ensuring security.

4.2 Abstraction:

Where: HotelService

Why: hide complex implementation detail

Snippet:

```
        public void bookRoom(int roomNumber, String name) {

        }
```

Explanation: User interacts with simple methods without knowing internal logic.

4.3 Inheritance:

Where: SaveThread extends Thread

Why: reuse existing code and establish relationships between classes

Snippet:

```
        public class SaveThread extends Thread {

            public void run() {

            }

        }
```

Explanation: Reuses Thread functionality for concurrent execution.

4.4 Polymorphism:

Where: display() method

Why: allow methods to behave differently based on the object or context

Snippet:

```
public void display() {  
    System.out.println("Room details");  
}
```

Explanation: Can be overridden for different room types.

4.5 Exception Handling:

Where: FileService class

Why: Manage runtime errors gracefully without crashing the program

Snippet:

```
try {  
    FileWriter writer = new FileWriter("file.txt");  
} catch(IOException e) {  
    System.out.println("Error saving file");  
}
```

Explanation: Prevents program crash during file operations.

4.6 Multithreading:

Where: SaveThread class

Why: perform multiple tasks simultaneously without affecting the main program flow

Snippet:

```
SaveThread t = new SaveThread(data);  
t.start();
```

Explanation: Runs file-saving in background to improve performance.

5. Implementation Highlights:

1. Booking Validation

```
if(!room.isBooked()) {  
    room.bookRoom();  
}
```

Justification: Prevents duplicate bookings

2. File Persistence

```
writer.write(data);
```

Justification: Stores booking data permanently.

3. Thread Execution

```
thread.start();
```

Justification: Improves responsiveness.

4. GUI Event Handling

```
addBtn.addActionListener(e -> {  
    hotel.addRoom(room, type);  
});
```

Justification: Handles user actions.

6. Testing & Error Handling:

- **Test Cases:**

- I. Add room successfully
- II. Book available room
- III. Cancel booking
- IV. View rooms

- **Edge Cases:**

- I. Booking already booked room
- II. Empty room list
- III. Invalid inputs

- **Failure Scenarios:**

- I. File write failure
- II. Incorrect user input
- III. Thread interruption

7. Git Workflow:

- Initial setup
- Added model classes
- Implemented service layer
- Added booking logic

- Implemented file handling
- Added threading
- Developed GUI
- Fixed bugs and improved UI

Explanation: The project was developed incrementally using meaningful commits. Each commit reflects a specific feature or improvement, ensuring proper version control and easy tracking of development progress.

8. Conclusion & Future Scope:

The Hotel Booking System successfully demonstrates the practical application of Object-Oriented Programming concepts in developing a real-world software solution. The system automates essential hotel operations such as room management, booking, and cancellation, thereby reducing manual effort and minimizing errors. By incorporating features like customer management, data persistence through file handling, and multithreading for background processing, the system ensures efficiency, reliability, and improved performance.

The use of a modular structure and separation of concerns enhances code readability, maintainability, and scalability. Additionally, the integration of a user-friendly interface using Java Swing improves the overall user experience. This project not only strengthens the understanding of core OOP principles but also provides a solid foundation for building more advanced and scalable applications in the future.

Future Scope:

- Integration with database
- Online booking system
- Payment gateway integration
- Admin dashboard
- Mobile application

1. Copy of the initial project proposal:

Project Title: Hotel Booking System using Java with Object-Oriented Programming

Problem Statement: Managing hotel room bookings manually can be difficult and time-consuming, especially when handling multiple rooms and customer reservations. Manual systems may lead to errors such as double bookings, incorrect records, or difficulty in tracking room availability. The Hotel Booking System aims to provide a simple digital solution to manage hotel reservations efficiently. The system allows users to add rooms, view available rooms, book rooms for customers, and cancel bookings. By using a Java-based application with Object-Oriented Programming concepts, the system helps organize booking records and improve efficiency.

Target User:

- Hotel reception staff
- Hotel managers
- Customers booking rooms
- Small hotel or guest house administrators

Core Features:

- Add and manage hotel rooms
- View available rooms
- Book a room for a customer
- Cancel room bookings
- Display room and booking details
- Track room availability status

OOP Concepts to be Used:

- **Abstraction:** The system will abstract hotel operations such as booking, cancelling, and managing rooms through service classes.
- **Inheritance:** Different types of rooms (e.g., standard, deluxe) can inherit common properties from a base room class.
- **Polymorphism:** Methods like displaying room details or calculating booking information can have different implementations.
- **Exception Handling:** Try-catch blocks will handle errors such as booking unavailable rooms or invalid customer input.
- **Collections / Threads:** Java `ArrayList` collection will be used to store room and booking records dynamically.

Proposed Architecture Description: The system will follow a **modular architecture** where the application is divided into separate components such as **model, service, and main packages**. The **model layer** represents entities like rooms and bookings. The **service layer** handles the main logic such as room booking, cancellation, and checking availability. The **main layer** manages user interaction through a menu-driven interface. This modular structure improves code organization, readability, and maintainability while clearly separating responsibilities within the system.

2. Copy of the review mail received (with APPROVED_MINOR)

9:51





soumadip.biswas@iemintern...
To You

Mar 10





Dear Soumi Chakraborty ,

Project: Hotel Booking System using Java with Object-Oriented Programming
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly identifies a common problem suitable for an OOP solution and outlines a well-defined scope. The proposed features are essential and appropriate for a hotel booking system, making it practical and relevant. The commitment to core OOP concepts like Abstraction, Inheritance, Polymorphism, Exception Handling, and Collections demonstrates a solid understanding of object-oriented design principles. The modular architecture (model, service, main layers) is well-articulated, promising good code organization, readability, and maintainability through clear separation of concerns.

:

